Secure GUI-Based Multi-Client Chat Application Using TCP
with Advanced Authentication and Session Hardening

Assignment 7

Submission Report

Submitted By	Deepak Singh Rawat
Enrollment No.	UU2409000093
Program	BCA
University	Uttaranchal University

Submitted To
ISEA Summer Internship 2026
ISEA under MeitY
(In affiliation with Tezpur University, Assam)

July 2026

Table of Contents

1. Objective

The objective of Assignment 7 is to upgrade the GUI-based multi-client TCP chat application from the earlier assignment into a more secure and resilient system. The work focuses on account protection, safer session handling, authentication control, and secure logging while preserving the same socket-based chat architecture.

The final application demonstrates a login window, protected user verification, duplicate-session blocking, failed-attempt tracking, and a lockout mechanism for repeated invalid access attempts. The server also records connection activity and security events without exposing plaintext passwords.

The key security goals are:

- User authentication using SHA-256 hashed passwords stored in users.json.
- Prevention of multiple simultaneous logins for the same account.
- Validation of usernames and connection parameters.
- Automatic lockout after five failed login attempts from the same IP.
- Centralized local logging in server_log.txt and security_log.txt.
- Packet-level verification of the application traffic in Wireshark.

2. Software Requirements

- Ubuntu 22.04 LTS / 24.04 LTS
- Python 3.x
- Tkinter for GUI development
- socket and threading modules
- hashlib for SHA-256 authentication
- JSON for credential storage
- Mininet for virtual network testing
- Wireshark for packet analysis
- Git and GitHub for version management

3. Network Topology

The experiment uses a simple Mininet topology with one server host and four client hosts. The server runs on h1, while h2, h3, h4, and h5 act as client nodes used for authentication, chat exchange, and packet capture testing.

Topology used in the experiment:

```
h1 ---- s1 ---- h2
        ├---- h3
        ├---- h4
        └---- h5
```

Mininet command used:

```
sudo mn --topo single,5
```

4. Security Architecture

The application follows a standard client-server architecture. Each client opens a TCP connection to the server, then performs local authentication using credentials matched against the SHA-256 password hashes stored on the server side.

- Authentication data is stored in users.json using hashed passwords only.
- The server rejects duplicate logins for any account that is already active.
- The server counts failed attempts from the same IP and starts a temporary lockout after repeated failures.
- The system logs normal and security-sensitive events separately.
- The GUI remains responsive while background threads receive messages and status updates.

5. Authentication and Session Hardening

The security mechanism has been designed to reduce common misuse patterns. A valid username and password are required for login, while invalid usernames or incorrect passwords are immediately rejected. Repeated failures from the same address trigger a timed lockout, which prevents brute-force guessing. The duplicate-login protection ensures that one user cannot create multiple live sessions at the same time.

The file users.json acts as a lightweight local credential database. The screenshots later in the report show the stored username-to-hash mapping and the resulting authentication behavior during successful, failed, and locked login attempts.

6. GUI Design

The interface is built with Tkinter and ttk widgets. The login window appears first and collects the username, password, server IP, and port. After authentication, the main chat window opens and provides a clean conversation view with online users, message input, and a connect/disconnect control.

- Login form with username, password, server IP, and port.
- Message area for chat history and system notifications.
- Online users panel for session visibility.
- Recipient selector for private messaging.
- Connect & Login button and a disconnect control.
- Responsive background receiving thread.

7. Server-Side Logging and Monitoring

The server records connection events, disconnections, invalid credential attempts, and account lockouts in log files. This makes it easier to debug session behavior and review access patterns later. The logs shown in the screenshots confirm that the system writes concise status entries while avoiding plaintext password exposure.

Appendix A. Evidence Screenshots

The screenshots below document the authentication workflow, the server logs, and network verification results.

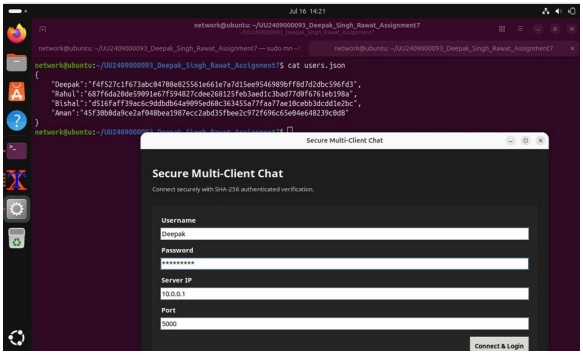

Figure 1: Secure login window with username, password, server IP, and port.

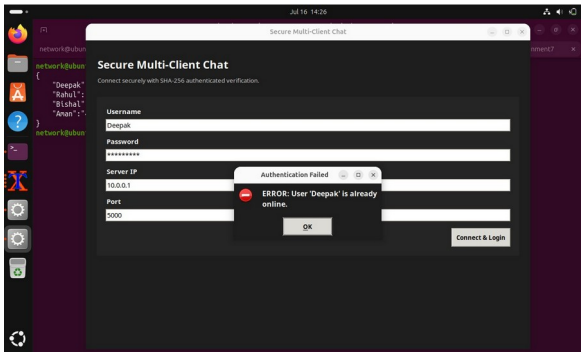

Figure 2: Duplicate login attempt rejected because the same user is already online.

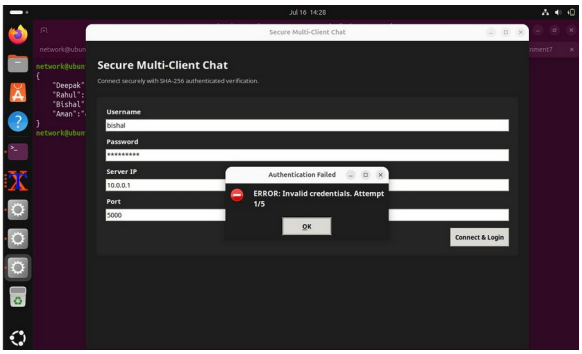

Figure 3: Invalid credentials trigger authentication failure.

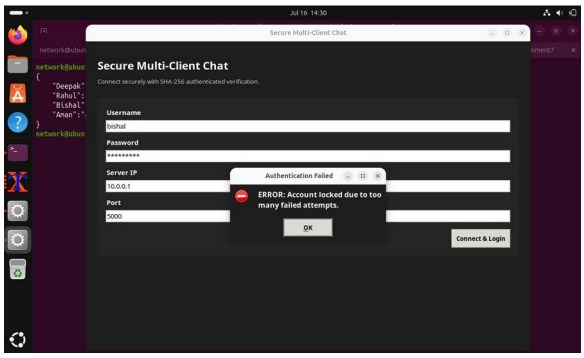

Figure 4: Account lockout after too many failed login attempts.

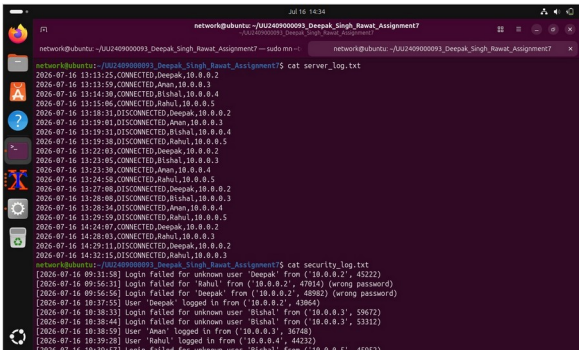

Figure 5: Server log and security log showing connection and authentication events.

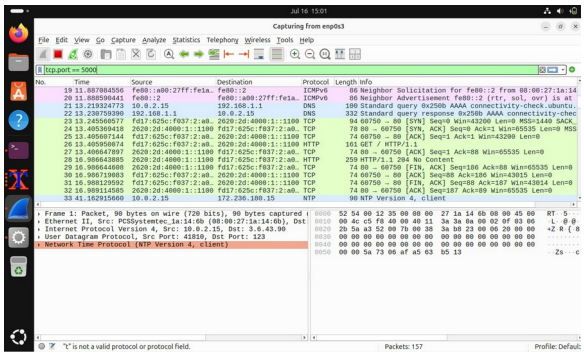

Figure 6: Wireshark capture showing TCP traffic on port 5000.

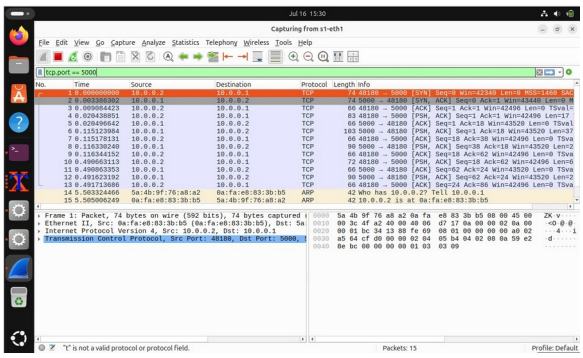

Figure 7: Wireshark view of the TCP connection handshake.

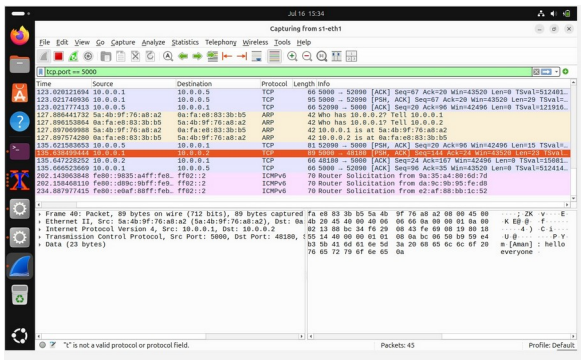

Figure 8: Packet capture showing application data and broadcast traffic.

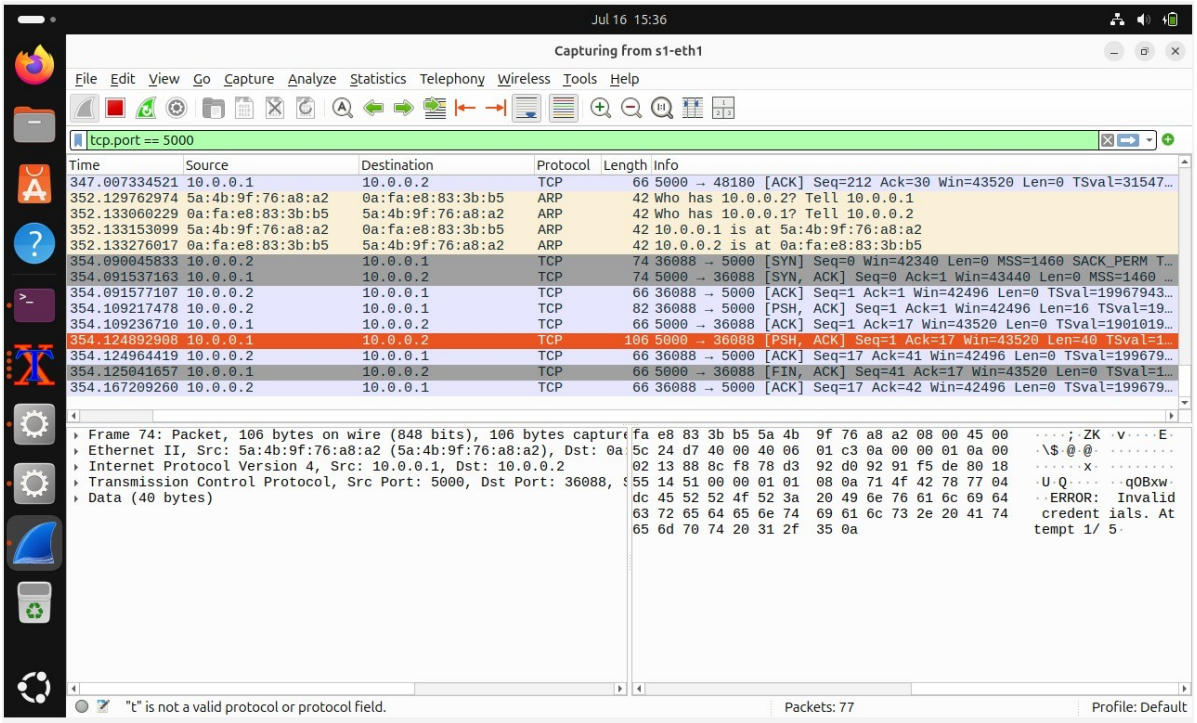

Figure 9: Packet capture showing private message delivery and session termination.

8. Testing Results

The application was tested in Mininet with one server and four clients. The tests confirmed that valid logins succeed, invalid attempts are blocked, duplicate access is denied, and the chat window stays responsive during message exchange.

Test case	Expected result	Observed result	Status
Login window	Accept valid username and connect	Client connected successfully to the TCP server	Pass
Duplicate login	Reject a session that is already active	Login blocked with duplicate-session message	Pass
Invalid credentials	Show authentication failure	Wrong password rejected and logged	Pass
Failed attempts	Count repeated failures	Attempt counter increased and lockout triggered	Pass
Broadcast message	Deliver message to all users	All connected clients received the broadcast	Pass
Private message	Deliver only to selected user	Only the intended recipient received the message	Pass

9. Wireshark Verification

Wireshark was used with the display filter tcp.port == 5000 to verify the TCP behavior of the secure chat application. The captures confirm the client connection sequence, application data transfer, and graceful termination of the session.

- TCP three-way handshake: SYN, SYN-ACK, ACK.
- Broadcast and private chat traffic visible as PSH, ACK data packets.
- Connection close visible through FIN, ACK exchange.
- The capture shows that the GUI client still uses the same socket-based communication logic as the terminal version.

10. Reflection Answers

Why should networking logic and GUI code be separated?

Separating the two keeps the application easier to maintain and debug. The socket logic can stay stable while the interface changes independently. It also allows the same backend to be reused with different frontends.

Why is a background thread required?

Network receive operations can block. A background thread keeps the GUI responsive while incoming messages are handled in parallel.

Which components from the earlier assignment were reused?

The TCP server, message routing, broadcast handling, private messaging, online user management, chat history, and server logging were reused.

How did the GUI improve usability?

The GUI removes the need to remember terminal commands for normal chatting. It provides a visible login screen, scrollable chat area, user list, and clear controls for sending and disconnecting.

Suggest one future enhancement.

A useful next step would be adding timestamps, message search, file transfer support, and stronger account recovery features.

11. Conclusion

Assignment 7 successfully transformed the GUI-based TCP chat application into a more secure system with password hashing, duplicate-login protection, failed-attempt tracking, and access lockout. The Mininet tests, GUI demonstration, and Wireshark captures confirmed that the application behaves correctly at both the user-interface level and the network level.

This assignment provided practical experience in Tkinter GUI development, threaded event-driven programming, session hardening, secure credential handling, and network packet verification.